

2024 Sept OPPE 1 - SET 2

Section 2 - Problem 1

Problem Statement

Problem statement:

- You need to write a function **count_positive_ignore_none**,
- that counts how many elements in a given list are:
 - Positive integers (greater than 0)
 - Ignoring all None values
 - Ignoring zero
- Return the total count of such positive integers.
- This is a function type question, you don't have to take input or print the output, just complete the required function definition.

Example and Explanation:

- Let our input be:
nums = [1, -2, 3, 0, None]
- Our argument here is a **list containing integers and None values**.
- We need to count how many numbers are:
 - **Positive integers (> than 0)**
 - **While ignoring None values**
 - **also ignoring 0**
- In our case:
 - 1 → Positive  (count = 1)
 - 2 → Negative  (ignore)
 - 3 → Positive  (count = 2)
 - 0 → Not positive  (ignore)
 - None → Ignored 
- So, the total no of **positive integers is 3**.
Hence, **the output should be 3**

Logic Visualization of our Problem

Our List with integers and possibly None values:

```
[0, -1, 2, None]
```

Iterate through every element of the list to find our answer.

Condition for positive integer?

```
int n > 0
```

Example:

7 > 0? True.

(Positive Integer)

-1 > 0? False

(Negative Integer)

How do we ignore null values?

```
# if num is not None:
```

We could either use:

- if statement
- List comprehension
- filter() function

Combine both our logics (many ways possible)

- if statement
- list comprehension
- single expression
return statement
using filter()

Concept of Iteration of elements

- In Python, **lists use zero-based indexing** to access elements by their position.
- A for loop allows visiting each element in a list sequentially.
- The loop variable holds the current element in each iteration.
- Combined with conditions, it can filter or process elements (e.g., count positives, skip None).

Example:

```
nums = [1, -2, 3, None, 4]

for num in nums:

    if num > 0:

        print(num)
```

For Loop: Visits each element sequentially using a loop variable.

Output: 1 3 4

Example:

```
i = 0

while i < len(nums):

    if nums[i] > 0:

        print(nums[i])

    i += 1
```

While Loop: Iterates using an index, giving more control over the loop.

Output: 1 3 4

How do we ignore the Null Values?

ADVANCED WAY: Using filter() function:

```
nums = [1, None, 2, None, 3]

cleaned = list(filter(lambda x: x is not None, nums))

print(cleaned)
```

SHORTEST WAY: Using list comprehension:

```
nums = [num for num in nums if num is not None]
```

MOST RISK FREE WAY: Using if conditional statement:

```
count = 0

for num in nums:

    if num is not None and num > 0:

        count += 1

return count
```

Our Solution Approach



The Explanatory Approach

Step-by-step implementation with if else statements for learning and understanding each concept



Alt way: Using filter() and lambda()

Using built in python functional programming techniques for efficiency



The Concise One-Liner Approach

Uses the single line List Comprehension Approach

Each solution produces identical results but demonstrates different Python programming techniques.

Solution 1: The Explanatory Approach

```
def count_positive_ignore_none(nums: list):  
    count = 0  
    for num in nums:  
        if num is not None and num > 0:  
            count += 1  
    return count
```

How this works:

- We use for loop to iterate through every element in the list
- Uses very basic if - else statements to see if conditions hold
- Increments count of variable 'c' if match is found.

The most beginner friendly way to solve this problem

Solution 2: Alt way Using Functional programming

```
def count_positive_ignore_none(nums: list):  
    return len(list(filter(lambda x: x is not None and x > 0, nums)))
```

How this works:

- **filter(function, iterable)** returns an iterator of elements in nums where the function returns **True**.
- The **lambda x: x is not None and x > 0** is a small anonymous function:
 - **x is not None** → skip None values
 - **x > 0** → include only positive numbers
 - **lambda arguments: expression** (a small nameless function in 1 line)
- So the filter keeps only numbers that are both not None and greater than 0

An alternative solution using functional programming

Solution 3: List Comprehension

```
def count_positive_ignore_none(nums: list):  
    return len([num for num in nums if num is not None and num > 0])
```



- This one-liner approach uses list comprehension to solve our problem which is a shorter way to build a list.
- It loops through the list, applies the conditions mentioned and returns a new filtered list
- **Trade-off: Slightly less readable** for beginners but very concise and Pythonic.

The Pythonic Optimization: One-liner solution using single expression solution
Result: Identical output, neater code

Solution comparison

Approach	Pros	Cons	Best For
Explanatory Approach	Highly readable, easy to debug, best for learning, clear variable names	More lines of code, multiple variables	Educational purposes, beginners, complex debugging
Functional Programming	Balanced readability and conciseness, single return statement, fewer variables	Requires understanding and practice of functions like <code>filter()</code> and <code>lambda()</code>	Production code, intermediate developers,
One-Liner Approach	Most concise, Pythonic, demonstrates mastery	Harder to debug, less readable for beginners, repeated calculation	Experienced developers, simple contexts

Key Takeaways

- 🔑 List Comprehension is a clean and efficient way to create a new list without None values.
- 🔑 filter() Function can remove None values or other “falsy” elements in a functional programming style.
- 🔑 while + remove() allows in-place removal of None, but may be slower for large lists.
- 🔑 for + if - else statement is the most risk- free, beginner friendly way to solve.
- 🔑 Choosing the right method depends on readability, performance, and whether you want to modify the original list or create a new one.